Lab 4- Data Normalization

Dylan Renard

Professor Brandon Higgs

Gene Expression and Data Visualization

Johns Hopkins University

September 25th, 2025

Lab 4:

In this lab, we will be working with a few data sets, each run on a different platform. The first data set is an R object generated from a 2-channel cDNA array that is called swirl. This data set is an experiment that was run on a zebrafish to study the early development. The data is named such because “swirl is a point mutant in the BMP2 gene that affects the dorsal/ventral body axis.” The objective of the experiment was to evaluate the transcript differences between wildtype zebrafish and those with this mutation. As I mentioned above, swirl is an R object, so the format and structure of this binary file has to be accessed through various R functions. If you type “swirl”, you will immediately see that there are attributes that make up this file (e.g. @maInfo) beyond the typical channel information. Included is metadata information that makes up the experimental parameters, in addition to the raw intensity data.

The second 2 data sets are raw intensity files – one from an Agilent platform and the other from an Affymetrix platform. Both of these files are on the course website. These are not R objects, rather the Agilent files are raw text files generated from the Agilent software and the Affymetrix files are raw binary files generated from the Affymetrix software.

Since both R objects and raw data files are typically what an analyst is given when asked to analyze an experiment, this lab will give you experience processing raw intensity files and normalizing them appropriately. This is typically the first step in conducting any microarray analysis, so it is important to make sure that the data is normalized appropriately before beginning any subsequent steps.

Data usage:

- Agilent (txt file) - GenePix_files.zip
- Affy (binary files)- HGU133plus_files.zip

Q1) Load the Libraries

Load the marray library and the swirl data set. This data set is an R metadata object, so there are multiple pieces of information (e.g., red/green background and foreground intensities, chip layout design, etc.) that

are stored in this R data object.

▼ Code

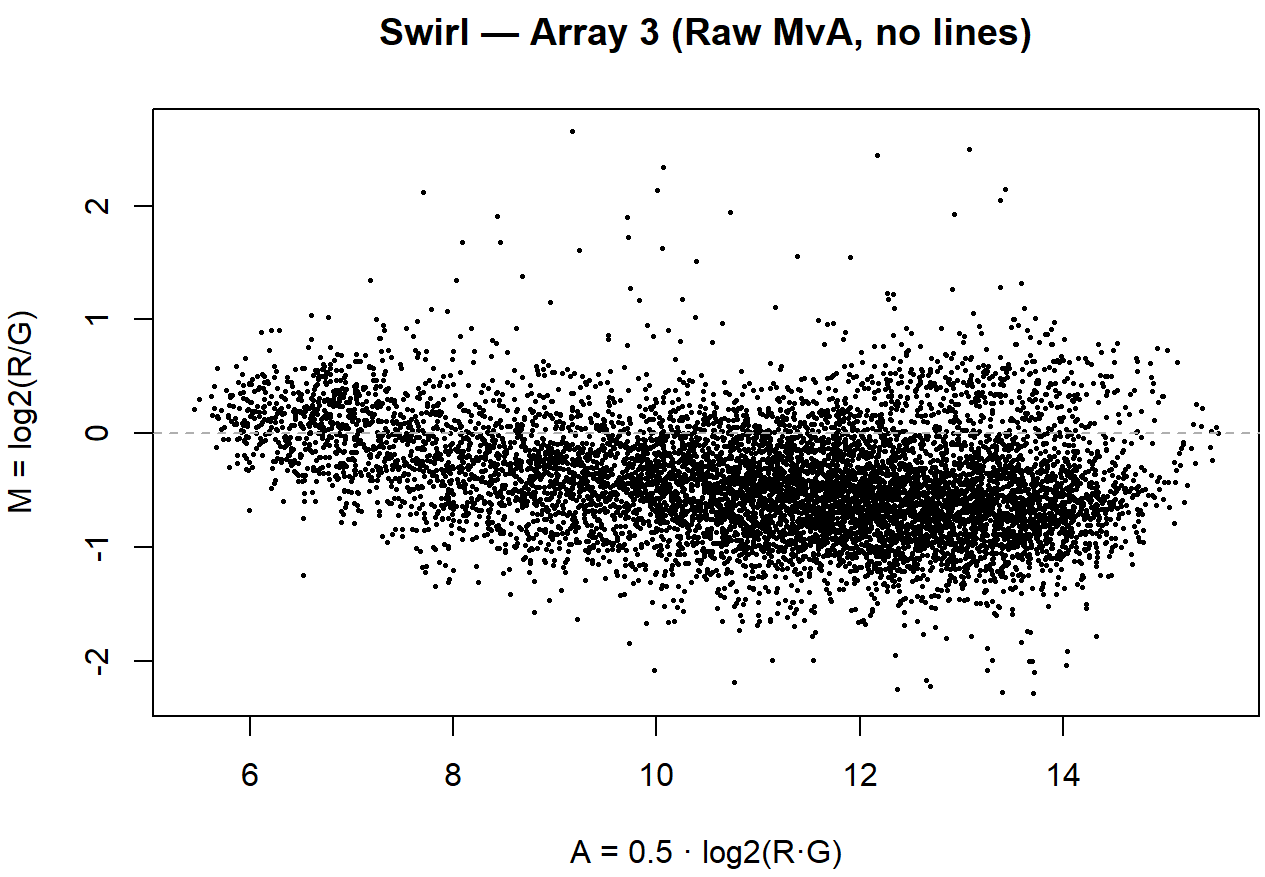
```
library(marray)
data(swirl)
library(ggplot2)
library(limma)
library(affydata)
library(affyPLM)
library(fpc)
library(DT)
```

Q2) Plot an MvA plot of array 3 without any stratified lines.

▼ Code

```
# Extract array 3
arr3 <- swirl[, 3]
A <- maA(arr3); M <- maM(arr3)

plot(A, M,
     pch=16, cex=0.38,
     xlab="A = 0.5 · log2(R·G)", ylab="M = log2(R/G)",
     main="Swirl — Array 3 (Raw MvA, no lines)")
abline(h=0, col="grey70", lty=2)
```



The raw MvA plot for swirl array 3 shows systematic dye bias.
Points are not centered at (M = 0) across the intensity range, and the spread increases with higher (A).
As discussed in Lecture 5a, these patterns reflect labeling and scanner artifacts, highlighting the need for normalization to remove intensity-dependent bias while preserving the true biological signal.

Q3/Q4) Global median localization normalization & plotting

Normalize array 3 by global median location normalization.

▼ Code

```
# Coerce to plain numeric vectors for ggplot
A <- as.vector(maA(arr3))
M <- as.vector(maM(arr3))

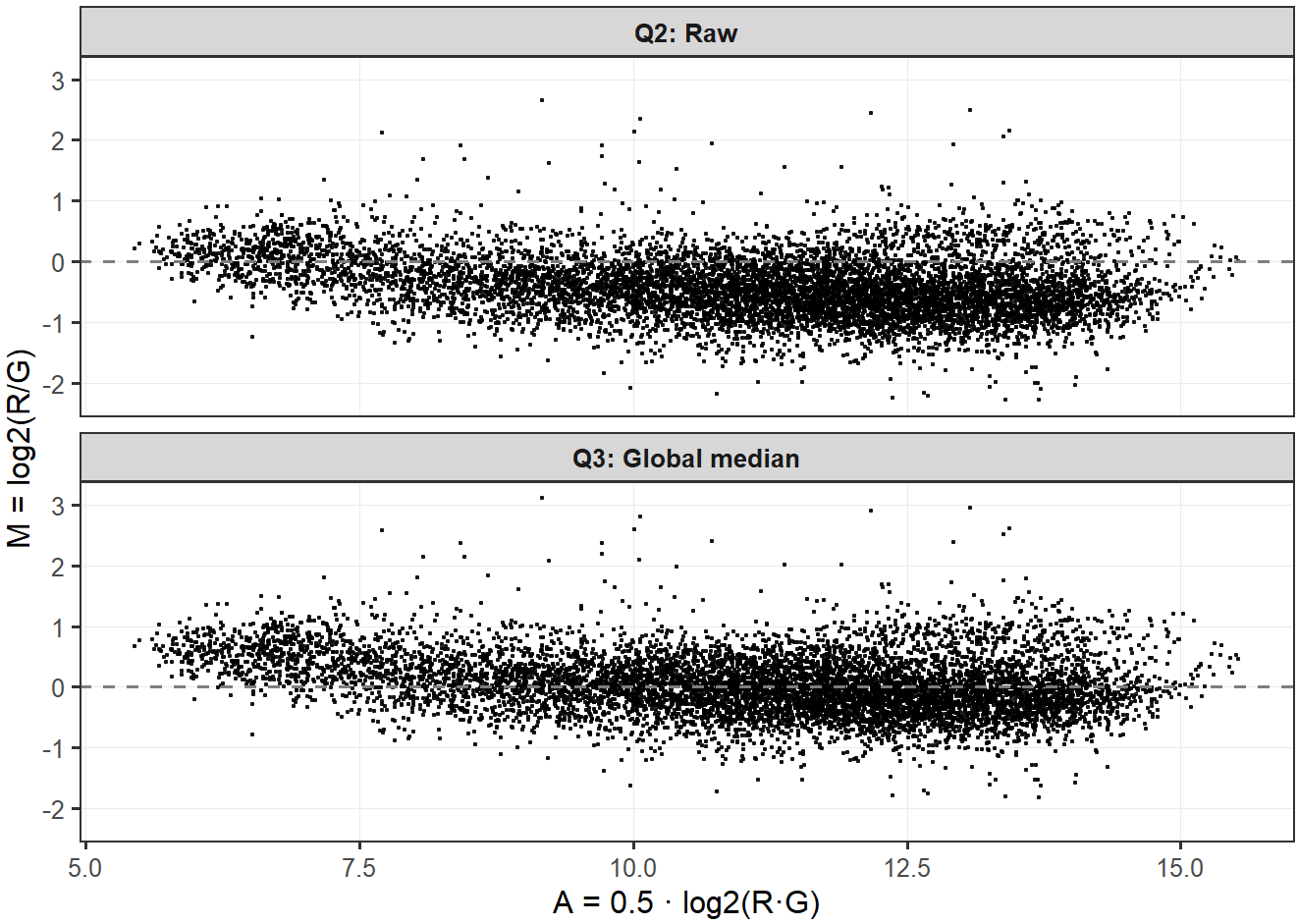
# Global median location normalization
med_raw  <- median(M, na.rm = TRUE)
M_glob   <- M - med_raw
med_glob <- median(M_glob, na.rm = TRUE)

# Tidy data
df <- rbind(
```

```
  data.frame(A = A, M = M,         state = "Q2: Raw"),
  data.frame(A = A, M = M_glob,    state = "Q3: Global median")
)
df$state <- factor(df$state, levels = c("Q2: Raw", "Q3: Global median"))

# Lock axes across facets
xlim_ <- range(df$A, na.rm = TRUE)
ylim_ <- range(df$M, na.rm = TRUE)

ggplot(df, aes(x = A, y = M)) +
  geom_point(size = 0.4, alpha = 0.7) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "grey50") +
  facet_wrap(~ state, nrow = 2, scales = "fixed") +  # vertical stack
  coord_cartesian(xlim = xlim_, ylim = ylim_) +
  labs(x = "A = 0.5 · log2(R·G)", y = "M = log2(R/G)") +
  theme_bw(base_size = 12) +
  theme(
    strip.text = element_text(face = "bold"),
    panel.grid.minor = element_blank(),
    panel.grid.major = element_line(linewidth = 0.2, color = "grey92")
  )
```



▼ Code

```
# Numeric check (like your base code)
cat("Median M (raw):   ", round(med_raw,  4), "\n")
```

```
Median M (raw):    -0.4602
```

▼ Code

```
cat("Median M (after): ", round(med_glob, 4), "\n")
```

```
Median M (after):  0
```

Q5) What is different between the normalized data and the non-normalized data?

In the raw MvA plot the cloud of points is shifted below (M = 0), reflecting an overall dye imbalance between the red and green channels.
Global median normalization simply recenters the distribution so that the median of (M) is zero.

As discussed in lecture, this adjustment corrects the global location bias but does not change the underlying curvature with intensity.
In other words, the main difference is that the normalized data are balanced around zero, while the overall shape of the plot remains the same as the raw data.

Q6) Lowess global intensity normalization and plotting.

Plot an MvA plot of the normalized array without the stratified lines or legend.

▼ Code

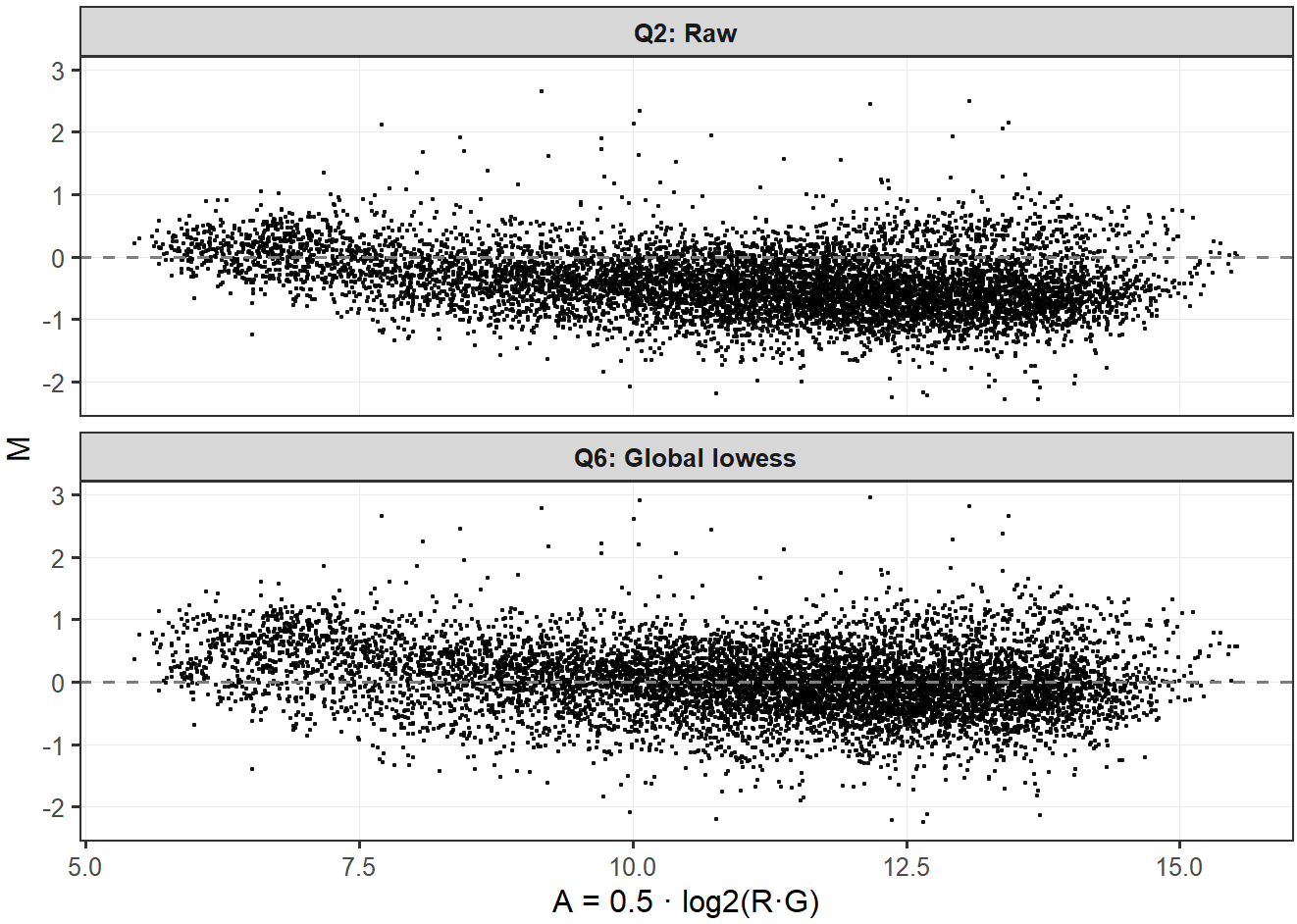
```
# Lowess fit, detrend, recenter
lo <- lowess(A, M, f = 2/3)
M_lowess <- M - lo$y
M_lowess <- M_lowess - median(M_lowess, na.rm = TRUE)

# Data frame for ggplot
df <- rbind(
  data.frame(A = A, M = M,        state = "Q2: Raw"),
  data.frame(A = A, M = M_lowess,  state = "Q6: Global lowess")
)
df$state <- factor(df$state, levels = c("Q2: Raw", "Q6: Global lowess"))

# Consistent axes
xlim_ <- range(df$A, na.rm=TRUE)
ylim_ <- range(df$M, na.rm=TRUE)

ggplot(df, aes(x = A, y = M)) +
```

```
  geom_point(size=0.4, alpha=0.7) +
  geom_hline(yintercept=0, linetype="dashed", color="grey50") +
  facet_wrap(~state, ncol=1, scales="fixed") +   # stacked vertically
  coord_cartesian(xlim = xlim_, ylim = ylim_) +
  labs(x = "A = 0.5 · log2(R·G)", y = "M") +
  theme_bw(base_size=12) +
  theme(
    strip.text = element_text(face="bold"),
    panel.grid.minor = element_blank(),
    panel.grid.major = element_line(linewidth=0.2, color="grey92")
  )
```



▼ Code

```
# Median checks
cat("Median M (raw):   ", round(median(M, na.rm=TRUE), 4), "\n")
```

```
Median M (raw):     -0.4602
```

▼ Code

```
cat("Median M (lowess): ", round(median(M_lowess, na.rm=TRUE), 4), "\n")
```

```
Median M (lowess):  0
```

Q7) Which of the two normalizations appears to be better for this array?

Between global median location normalization (Q3) and global lowess normalization (Q6), the **lowess normalization is clearly better** for this array.

- **Global median** only shifts the distribution so the overall dye balance is corrected, but the intensity-dependent curvature remains in the MvA plot.
- **Global lowess** removes that curvature by adjusting (M) as a function of (A), so the point cloud is symmetrically centered around (M=0) across the full intensity range.

As emphasized in lecture, lowess normalization is preferred because it corrects both the global dye bias and the local, intensity-dependent effects, giving a truer picture of biological variation.

Q8) loading GenePix files

Now we would like to read in raw GenePix files for 2 cDNA arrays that represent 2 patient samples. Go to the course website and retrieve the compressed file called 'GenePix files'. Open it up and put the contents in a directory. Now using the sample code below, read in the 2 array files.

> dir.path <- "C:\Documents and Settings\higgsb\Desktop\" a.cdna <- read.GenePix(path=dir.path,name.Gf = "F532 Median",name.Gb ="B532 Median", name.Rf = "F635 Median", name.Rb = "B635 Median",name.W ="Flags")

▼ Code

```
# 2. Read in both arrays
# The function will automatically grab all .gpr files in dir.path

dir.path <- "C:/Users/drena/OneDrive/Desktop/CodingRepos/GeneExpression/Lab4/GenePix_files"
# Read both arrays into an a.cdna object
a.cdna <- read.GenePix(
  path    = dir.path,
  # if your converted files end in "_utf8.gpr", use that pattern
  files   = list.files(dir.path, pattern = "_utf8\\.gpr$", full.names = FALSE),
  name.Gf = "F532 Median",
  name.Gb = "B532 Median",
  name.Rf = "F635 Median",
  name.Rb = "B635 Median",
  name.W  = "Flags"
)
```

```
Reading ...
C:/Users/drena/OneDrive/Desktop/CodingRepos/GeneExpression/Lab4/GenePix_files/patient1_utf8.gpr
Reading ...
C:/Users/drena/OneDrive/Desktop/CodingRepos/GeneExpression/Lab4/GenePix_files/patient2_utf8.gpr
```

▼ Code

```
# Quick check
a.cdna
```

```
An object of class "marrayRaw"
@maRf

C:/Users/drena/OneDrive/Desktop/CodingRepos/GeneExpression/Lab4/GenePix_files/patient1_utf8.gpr
[1,]
890
[2,]
1420
[3,]
4420
[4,]
102
[5,]
84

C:/Users/drena/OneDrive/Desktop/CodingRepos/GeneExpression/Lab4/GenePix_files/patient2_utf8.gpr
[1,]
2416
[2,]
3077
[3,]
8038
[4,]
73
[5,]
177
19195 more rows ...

@maGf

C:/Users/drena/OneDrive/Desktop/CodingRepos/GeneExpression/Lab4/GenePix_files/patient1_utf8.gpr
[1,]
2747
[2,]
3990
[3,]
9717
[4,]
400
[5,]
```

```
266

C:/Users/drena/OneDrive/Desktop/CodingRepos/GeneExpression/Lab4/GenePix_files/patient2_utf8.gpr
[1,]
2925
[2,]
3620
[3,]
8119
[4,]
232
[5,]
454
19195 more rows ...

@maRb

C:/Users/drena/OneDrive/Desktop/CodingRepos/GeneExpression/Lab4/GenePix_files/patient1_utf8.gpr
[1,]
49
[2,]
49
[3,]
53
[4,]
50
[5,]
53

C:/Users/drena/OneDrive/Desktop/CodingRepos/GeneExpression/Lab4/GenePix_files/patient2_utf8.gpr
[1,]
60
[2,]
58
[3,]
63
[4,]
61
[5,]
63
19195 more rows ...

@maGb

C:/Users/drena/OneDrive/Desktop/CodingRepos/GeneExpression/Lab4/GenePix_files/patient1_utf8.gpr
[1,]
94
[2,]
94
[3,]
102
```

```
[4,]
98
[5,]
106

C:/Users/drena/OneDrive/Desktop/CodingRepos/GeneExpression/Lab4/GenePix_files/patient2_utf8.gpr
[1,]
172
[2,]
158
[3,]
149
[4,]
148
[5,]
179
19195 more rows ...

@maW

C:/Users/drena/OneDrive/Desktop/CodingRepos/GeneExpression/Lab4/GenePix_files/patient1_utf8.gpr
[1,]
0
[2,]
0
[3,]
0
[4,]
0
[5,]
0

C:/Users/drena/OneDrive/Desktop/CodingRepos/GeneExpression/Lab4/GenePix_files/patient2_utf8.gpr
[1,]
0
[2,]
0
[3,]
0
[4,]
-50
[5,]
0
19195 more rows ...

@maLayout
An object of class "marrayLayout"
@maNgr
[1] 12

@maNgc
```

```
[1] 4

@maNsr
[1] 20

@maNsc
[1] 20

@maNspots
[1] 19200

@maSub
[1] TRUE TRUE TRUE TRUE TRUE
19195 more elements ...

@maPlate
[1] 1 1 1 1 1
50 Levels: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 ... 50
19195 more elements ...

@maControls
[1] probes probes probes probes probes
Levels: Buffer probes
19195 more elements ...

@maNotes
character(0)


@maGnames
An object of class "marrayInfo"
@maLabels
[1] "36894"   "N/A"     "5548382" "N/A"     "29305"
19195 more elements ...

@maInfo
              ID                                      Name
X36894     36894        R34425:Data not found::::10:34049
N.A          N/A      BM970759:Data not found::::10:33873
X5548382 5548382 BM549033:Hs.159410:MOCS3:20:20q13.13:3:3
N.A.1        N/A        F12650:Data not found::::10:33438
X29305     29305     R14519:Hs.97393:ALMS1:2:2p13:0:29305
19195 more rows ...

@maNotes
[1] ""


@maTargets
An object of class "marrayInfo"
@maLabels
```

```
character(0)

@maInfo
data frame with 0 columns and 0 rows

@maNotes
character(0)


@maNotes
[1] "GenePix Data"
```

▼ Code

```
slotNames(a.cdna)
```

```
[1] "maRf"      "maGf"      "maRb"      "maGb"      "maW"       "maLayout"
[7] "maGnames"  "maTargets" "maNotes"
```

Q9) ploting the normalization

Using the a.cdna object, which is analogous to the swirl metadata object, normalize both arrays and provide MvA plots for each array normalized by the following 3 methods: no normalization, print-tip lowess normalization, and scale print-tip normalization using the MAD. Hint: use the par(mfrow=c(3,1)) function to put the 3 plots for a single patient array on the same page.

▼ Code

```
# Helper to compute raw A/M from a.cdna for one array (j)
compute_AM_raw <- function(a, j) {
  Rf <- as.matrix(a@maRf)[, j]
  Gf <- as.matrix(a@maGf)[, j]
  Rf[Rf <= 0] <- NA; Gf[Gf <= 0] <- NA
  A <- 0.5 * (log2(Rf) + log2(Gf))
  M <- log2(Rf) - log2(Gf)
  list(A = A, M = M)
}

# Build tidy df for one array (patient) with 3 methods
df_for_array <- function(a, j, sample_name) {
  # 1) No normalization
  raw <- compute_AM_raw(a, j)
  df_none <- data.frame(A = raw$A, M = raw$M,
                        method = "No normalization", sample = sample_name)

  # 2) Print-tip LOWESS (within-array)
  a_pt <- maNorm(a[, j, drop = FALSE], norm = "printTipLoess")
  # maNorm updates M; A is unchanged for within-array normalization
  df_pt <- data.frame(A = raw$A, M = as.vector(a_pt@maM),
```

```
                      method = "Print-tip LOWESS", sample = sample_name)

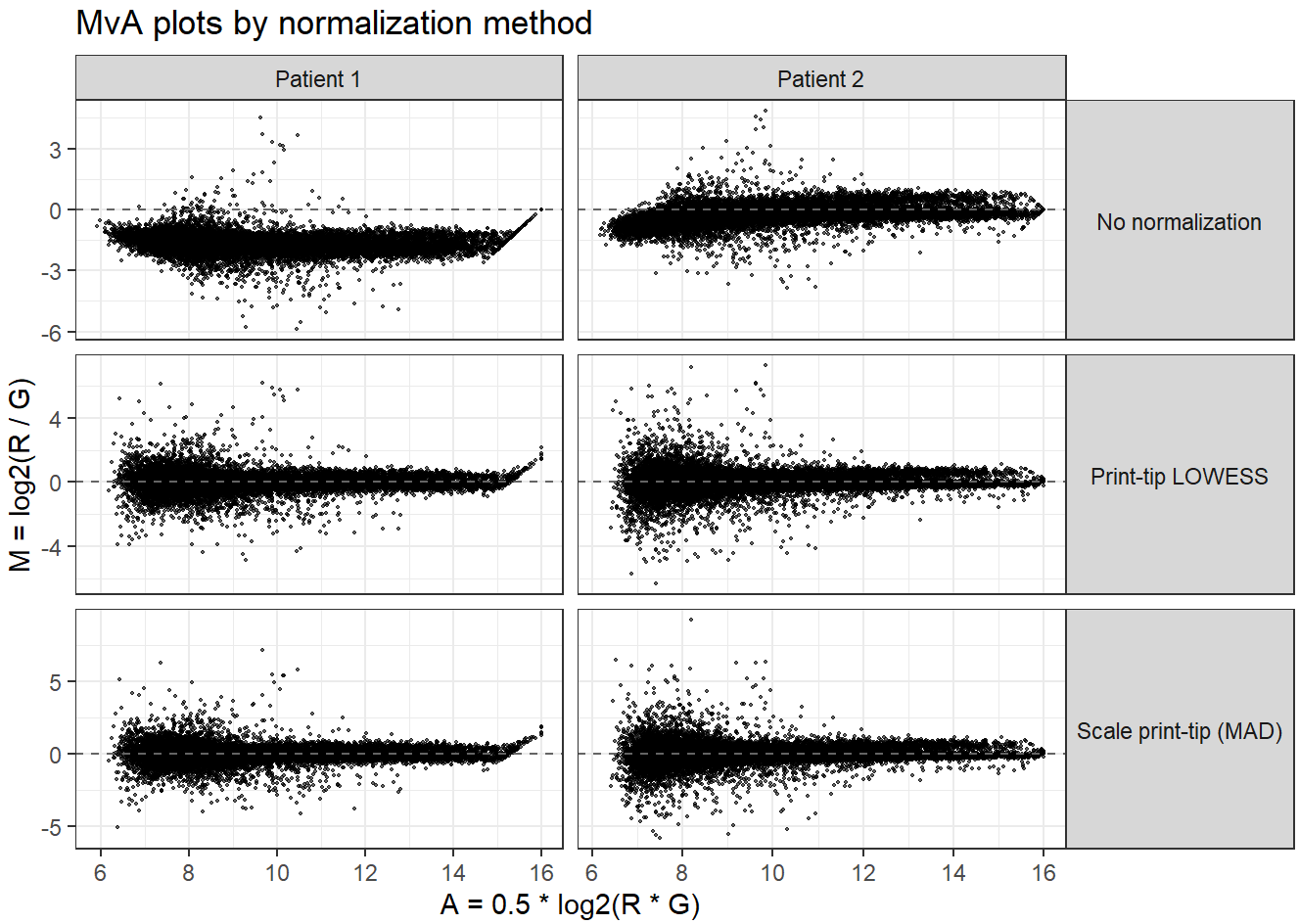
  # 3) Scale print-tip (MAD)
  a_sc <- maNorm(a[, j, drop = FALSE], norm = "scalePrintTipMAD")
  df_sc <- data.frame(A = raw$A, M = as.vector(a_sc@maM),
                      method = "Scale print-tip (MAD)", sample = sample_name)

  rbind(df_none, df_pt, df_sc)
}

# Build data for both arrays (assumes exactly 2 arrays in a.cdna)
df_all <- rbind(
  df_for_array(a.cdna, 1, "Patient 1"),
  df_for_array(a.cdna, 2, "Patient 2")
)

# Plot: 3 methods (rows) × 2 patients (columns)
ggplot(df_all, aes(x = A, y = M)) +
  geom_point(size = 0.3, alpha = 0.6) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "grey40") +
  facet_grid(method ~ sample, scales = "free") +
  labs(
    title = "MvA plots by normalization method",
    x = "A = 0.5 * log2(R * G)",
    y = "M = log2(R / G)"
  ) +
  theme_bw(base_size = 11) +
  theme(strip.text.y = element_text(angle = 0))
```

```
Warning: Removed 1610 rows containing missing values or values outside the scale range
(`geom_point()`).
```



Q10) normalized dataframe

Finally, we would like to create a data matrix that can be written out to a file with 19,200 rows and 2 columns (i.e. each patient array). Using the functions maM(), maGnames(), and maLabels(), figure out how to create the data matrix, get the probe IDs, and assign the probe IDs to the row names. Do this for the 2 normalized metadata objects that you created in #9 above (don't worry about the un-normalized data matrix).

▼ Code

```
## 1) Normalize BOTH arrays at once (same as Q9 methods)
a_pt <- maNorm(a.cdna, norm = "printTipLoess")
a_sc <- maNorm(a.cdna, norm = "scalePrintTipMAD")

## 2) Extract M as matrices (spots x arrays)
M_pt <- as.matrix(maM(a_pt))
M_sc <- as.matrix(maM(a_sc))

## 3) Get probe IDs and assign as row names
gnames    <- maGnames(a.cdna)          # gene/spot annotations
probe_ids <- maLabels(gnames)          # character vector of IDs
```

```
stopifnot(length(probe_ids) == nrow(M_pt), ncol(M_pt) == 2)

rownames(M_pt) <- probe_ids
rownames(M_sc) <- probe_ids

## 4) Give informative column names (2 patient arrays)
colnames(M_pt) <- c("Patient1", "Patient2")
colnames(M_sc) <- c("Patient1", "Patient2")

## 6) (Optional) write out
# write.table(M_pt, file = "MvA_printTipLoess_19200x2.txt",
#             sep = "\t", quote = FALSE, row.names = TRUE)
# write.table(M_sc, file = "MvA_scalePrintTipMAD_19200x2.txt",
#             sep = "\t", quote = FALSE, row.names = TRUE)

## Return objects in your environment for grading:
## Return only dims and heads for grading
cat("M_pt dimensions:", paste(dim(M_pt), collapse = " x "), "\n")
```

```
M_pt dimensions: 19200 x 2
```

▼ Code

```
print(head(M_pt))
```

```
          Patient1   Patient2
36894   -0.03311015  0.1505862
N/A      0.13049475  0.1713035
5548382  0.52435894  0.3575525
N/A     -0.78083988 -1.5229202
29305   -0.49450905 -0.6409908
28821   -0.83639640 -0.7880698
```

▼ Code

```
cat("\nM_sc dimensions:", paste(dim(M_sc), collapse = " x "), "\n")
```

```
M_sc dimensions: 19200 x 2
```

▼ Code

```
print(head(M_sc))
```

```
          Patient1   Patient2
36894   -0.04261438  0.1453677
N/A      0.16795314  0.1653669
5548382  0.67487566  0.3451614
```

```
N/A      -1.00497921 -1.4701431
29305    -0.63645739 -0.6187771
28821    -1.07648317 -0.7607591
```

Q11) Load the following libraries: affy, limma, affydata, affyPLM, and fpc.

Done in cell at Q1

Q12)

Now we would like to read in 3 raw Affymetrix .CEL files and normalize them with 2 different algorithms. These 3 arrays represent 3 normal healthy subjects that should have similar expression profiles. They are on the course website in the compressed file called Affymetrix .CEL files. Use the following code below to read in a metadata object for the 3 arrays (dir.path should be the same as above).

> fns <- sort(list.celfiles(path=dir.path,full.names=TRUE)) data.affy <- ReadAffy(filenames=fns,phenoData=NULL)

▼ Code

```
dir.path <- "C:/Users/drena/OneDrive/Desktop/CodingRepos/GeneExpression/Lab4/HGU133plus_files"
# 2) Find the 3 CEL files and read them
fns <- sort(list.celfiles(path = dir.path, full.names = TRUE))
stopifnot(length(fns) == 3)

data.affy <- ReadAffy(filenames = fns, phenoData = NULL)

# (Optional) make sample names short/nice
sampleNames(data.affy) <- gsub("\\.CEL$","", basename(fns), ignore.case = TRUE)

# 3) Normalize with two algorithms

## RMA (background correction + quantile normalization + median polish; returns log2)
eset_rma  <- rma(data.affy)               # ExpressionSet (log2)
```

```
Warning: replacing previous import 'AnnotationDbi::head' by 'utils::head' when
loading 'hgu133plus2cdf'

Warning: replacing previous import 'AnnotationDbi::tail' by 'utils::tail' when
loading 'hgu133plus2cdf'


Background correcting
Normalizing
Calculating Expression
```

▼ Code

```
mat_rma   <- exprs(eset_rma)              # matrix [probesets x 3]

## MAS5 (returns linear scale by default; convert to log2 for comparability)
eset_mas5 <- mas5(data.affy)              # ExpressionSet (linear)
```

```
background correction: mas
PM/MM correction : mas
expression values: mas
background correcting...done.
54675 ids to be processed
|                    |
|####################|
```

▼ Code

```
mat_mas5  <- log2(pmax(exprs(eset_mas5), 1))  # log2-transform; guard zeros

# 4) Return compact outputs (dims + heads) for grading/checks
cat("RMA matrix dims:", paste(dim(mat_rma), collapse=" x"), "\n")
```

```
RMA matrix dims: 54675 x3
```

▼ Code

```
print(head(mat_rma))
```

```
           normal1  normal2  normal3
1007_s_at 6.732547 6.772589 6.950047
1053_at   6.986680 6.986680 6.986680
117_at    7.334083 9.792765 9.794139
121_at    7.179351 7.439034 7.677756
1255_g_at 3.423446 3.423446 3.423446
1294_at   8.402456 8.402456 8.402456
```

▼ Code

```
cat("\nMAS5 (log2) matrix dims:", paste(dim(mat_mas5), collapse=" x"), "\n")
```

```

MAS5 (log2) matrix dims: 54675 x3
```

▼ Code

```
print(head(mat_mas5))
```

```
            normal1   normal2   normal3
1007_s_at  8.971711  8.648733  9.027732
1053_at    8.723186  9.212139  9.142913
117_at     8.793837 11.674111 11.733565
121_at     8.981001  8.952787  9.514138
1255_g_at  6.663684  6.026078  5.733717
1294_at   10.447909 10.200471 10.222631
```

Q13) Data matricies for Affy data

Using the function: expresso in addition to exprs(), create the normalized data matrices with 54,675 rows and 3 columns for the 2 different normalization algorithms.

Be sure to use normalize.method="quantiles", summary.method="medianpolish", and for RMA: pmcorrect.method="pmonly" MAS: pmcorrect.method="mas"

▼ Code

```
# ---- Q13: expresso normalization with caching ----
if (!requireNamespace("affy", quietly = TRUE)) {
  if (!requireNamespace("BiocManager", quietly = TRUE)) install.packages("BiocManager")
  BiocManager::install("affy")
}
library(affy)

# Set your save paths
rma_file <- "mat_rma_exp.rds"
mas_file <- "mat_mas_exp.rds"

if (file.exists(rma_file) && file.exists(mas_file)) {
  cat("Loading cached RDS results...\n")
  mat_rma_exp <- readRDS(rma_file)
  mat_mas_exp <- readRDS(mas_file)
} else {
  cat("Running expresso normalizations (may take a while)...\n")

  # RMA
  t_rma <- system.time({
    eset_rma_exp <- expresso(
      data.affy,
      bgcorrect.method = "rma",
      normalize.method = "quantiles",
      pmcorrect.method = "pmonly",
      summary.method   = "medianpolish"
    )
  })
  mat_rma_exp <- exprs(eset_rma_exp); rm(eset_rma_exp); gc()
  saveRDS(mat_rma_exp, rma_file)
```

```
  # MAS
  t_mas <- system.time({
    eset_mas_exp <- expresso(
      data.affy,
      bgcorrect.method = "mas",
      normalize.method = "quantiles",
      pmcorrect.method = "mas",
      summary.method   = "medianpolish"
    )
  })
  mat_mas_exp <- exprs(eset_mas_exp); rm(eset_mas_exp); gc()
  saveRDS(mat_mas_exp, mas_file)

  cat("RMA time:", t_rma["elapsed"], "sec\n")
  cat("MAS time:", t_mas["elapsed"], "sec\n")
}
```

```
Loading cached RDS results...
```

▼ Code

```
# Optional: ensure MAS on log2 scale
if (max(mat_mas_exp, na.rm = TRUE) > 100) {
  mat_mas_exp <- log2(pmax(mat_mas_exp, 1))
}

# Add sample names if available
if (!is.null(sampleNames(data.affy))) {
  colnames(mat_rma_exp) <- sampleNames(data.affy)
  colnames(mat_mas_exp) <- sampleNames(data.affy)
}

# Return dims + heads only
cat("RMA (expresso) dims:", paste(dim(mat_rma_exp), collapse = " x "), "\n")
```

```
RMA (expresso) dims: 54675 x 3
```

▼ Code

```
print(head(mat_rma_exp))
```

```
           normal1  normal2  normal3
1007_s_at 6.732547 6.772589 6.950047
1053_at   6.986680 6.986680 6.986680
117_at    7.334083 9.792765 9.794139
121_at    7.179351 7.439034 7.677756
1255_g_at 3.423446 3.423446 3.423446
1294_at   8.402456 8.402456 8.402456
```

▼ Code

```
cat("\nMAS (expresso) dims:", paste(dim(mat_mas_exp), collapse = " x "), "\n")
```

```
MAS (expresso) dims: 54675 x 3
```

▼ Code

```
print(head(mat_mas_exp))
```

```
           normal1  normal2  normal3
1007_s_at 6.413600 6.506238 6.555331
1053_at   6.875463 6.875463 6.832845
117_at    6.296964 9.376086 9.376086
121_at    6.882140 6.992261 7.372421
1255_g_at 3.790706 3.790706 3.790706
1294_at   8.041243 8.041243 8.041243
```

Q14) Normalization method comparison

Now use the cor() function to calculate the correlation between the 3 arrays for both normalized data matrices. Since these 3 subjects are all healthy normal individuals, we would expect to see somewhat good correlation structure between them all when looking across all genes on the array. Which normalization method has a higher overall correlation structure for these 3 normal healthy subjects? Show how you arrived at this answer.

▼ Code

```
# Align rows (safety)
common_ids <- intersect(rownames(mat_rma_exp), rownames(mat_mas_exp))
mat_rma <- mat_rma_exp[common_ids, , drop = FALSE]
mat_mas <- mat_mas_exp[common_ids, , drop = FALSE]

# Correlation matrices (Pearson)
cor_rma <- cor(mat_rma, use = "pairwise.complete.obs", method = "pearson")
cor_mas <- cor(mat_mas, use = "pairwise.complete.obs", method = "pearson")

# Average off-diagonal correlation
offdiag_mean <- function(M) mean(M[upper.tri(M)], na.rm = TRUE)
mean_rma <- offdiag_mean(cor_rma)
mean_mas <- offdiag_mean(cor_mas)

# Helper: pretty DT for a correlation matrix
cor_dt <- function(M, title = "Correlation matrix") {
  Mround <- round(M, 3)
  df <- data.frame(Sample = rownames(Mround), Mround, check.names = FALSE)
```

```
  # Build cuts from the observed range and make values = cuts + 1
  rng   <- range(as.matrix(Mround), na.rm = TRUE)
  cuts  <- seq(rng[1], rng[2], length.out = 7)[-c(1, 7)]   # 5 cuts
  vals  <- colorRampPalette(c("#f7fbff", "#6baed6", "#08306b"))(length(cuts) + 1)  # 6 colors

  dt <- datatable(
    df,
    rownames = FALSE,
    caption = htmltools::tags$caption(style = "caption-side: top; text-align: left;", title),
    options = list(dom = "t", scrollX = TRUE)
  )

  # Apply gradient to numeric columns only
  num_cols <- colnames(df)[sapply(df, is.numeric)]
  for (cn in num_cols) {
    dt <- formatStyle(dt, cn, backgroundColor = styleInterval(cuts, vals))
  }
  dt
}

# Build DT tables
dt_rma <- cor_dt(cor_rma, title = "RMA (expresso): 3×3 Pearson correlations")
dt_mas <- cor_dt(cor_mas, title = "MAS (expresso): 3×3 Pearson correlations")

# Summary DT (which normalization has higher overall correlation?)
winner <- if (is.finite(mean_rma) && is.finite(mean_mas)) {
  if (mean_rma > mean_mas) "RMA" else if (mean_mas > mean_rma) "MAS" else "Tie"
} else NA_character_

summary_df <- data.frame(
  Method = c("RMA (expresso)", "MAS (expresso)"),
  Mean_OffDiag_r = round(c(mean_rma, mean_mas), 4),
  stringsAsFactors = FALSE
)

dt_summary <- datatable(
  summary_df,
  rownames = FALSE,
  caption = htmltools::tags$caption(
    style = "caption-side: top; text-align: left;",
    paste0("Overall correlation summary (winner: ", winner, ")")
  ),
  options = list(dom = "t")
)

# Display (in Quarto/knitr these will render one after another)
dt_summary
```

Overall correlation summary (winner: RMA)

Method	Mean_OffDiag_r
RMA (expresso)	0.9813
MAS (expresso)	0.915

▼ Code

```
dt_rma
```

RMA (expresso): 3×3 Pearson correlations

Sample	normal1	normal2	
normal1	1	0.977	0.976
normal2	0.977	1	0.991
normal3	0.976	0.991	1

▼ Code

```
dt_mas
```

MAS (expresso): 3×3 Pearson correlations

Sample	normal1	normal2	
normal1	1	0.896	0.9
normal2	0.896	1	0.949
normal3	0.9	0.949	1

Q14 Interpretation/Explanation

After normalizing the three Affymetrix arrays with both RMA and MAS, I calculated pairwise Pearson correlations across all 54,675 genes. Because all three subjects are healthy, the expectation from lecture is that their expression profiles should be broadly similar, yielding strong correlations regardless of normalization. However, the method of normalization can influence how well this similarity is preserved. With RMA (background correction by RMA, quantile normalization, PM-only correction, and median polish summarization), the average off-diagonal correlation was 0.9813, compared to 0.9150 for MAS (MAS background correction and PM correction with the same normalization/summarization). The higher correlation values under RMA indicate that its quantile normalization and robust summarization produce more consistent expression measures across arrays, which is exactly the advantage we discussed in lecture.

Thus, RMA provides the stronger overall correlation structure for these three normal healthy subjects, making it the preferable method in this context.